

LAPORAN KUIS

MAULANA IKHSAN 5025241163

Hari, tanggal : Selasa, 17 maret 2026
Kelas : B
Tahun ajar : 2025/2026

1. TUGAS

Buatlah resume dari pengerjaan soal kuis

SPOJ yaitu "TPC07 - Change"

Link soal : <https://www.spoj.com/problems/TPC07/>

Ss soal :

TPC07 - Change

no tags

How many ways are there to pay n cents? We assume that the payment must be made with pennies (1 cent), nickels (5 cents), dimes (10 cents), quarters (25 cents), and half-dollars (50 cents).

For example, there are four ways to pay 13 cents, namely (13 pennies), (2 nickels, 3 pennies), (1 nickel, 8 pennies), and (1 dime, 3 pennies).

Input

The input will contain multiple test cases. Each test case contains a single line with a single integer n ($1 \leq n \leq 1000000000$).

The input will be terminated by the end of file.

Output

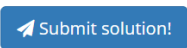
For each input integer n , output how many ways are there to pay n cents in a single line.

Sample Input

```
13
1000000000
```

Sample Output

```
4
66666793333412666685000001
```

 Submit solution!

2. DETAIL SOAL

Deskripsi :

How many ways are there to pay n cents? We assume that the payment must be made with pennies (1 cent), nickels (5 cents), dimes (10 cents), quarters (25 cents), and half-dollars (50 cents).

For example, there are four ways to pay 13 cents, namely (13 pennies), (2 nickels, 3 pennies), (1 nickel, 8 pennies), and (1 dime, 3 pennies).

Format input :

The input will contain multiple test cases. Each test case contains a single line with a single integer n ($1 \leq n \leq 1000000000$).

Format output :

For each input integer n , output how many ways are there to pay n cents in a single line.

Example :

Input:

13

100000000

Output:

4

66666793333412666685000001

3. ANALISIS SOAL

Pada soal TPC07 ini merupakan variasi dari problem seperti CBANK yang kemarin sudah ditugaskan, tetapi diberi sedikit perbedaan pada syarat dan juga constraint. Approach yang akan digunakan merupakan approach iterative untuk mencari solusi optimal dikarenakan approach normal dynamic programming akan TLE(kalo normal kita bakal bikin array dp ukuran 10^9 yang mana bakal error).

Berdasarkan buku “Concrete mathematics – a foundation for computer science” by Knuth, dijelaskan pada bab 7 yaitu generating function, pada subbab 7.1 mengenai analisis pada masalah domino dan change, halaman 327-330 didapatkan rumus untuk penyusunan :

- Sebuah bilangan P dengan 1 sen : $p = 1/(1 - z)$
- Sebuah bilangan N dengan P sen : $n = p/(1 - z^p)$
- Sebuah bilangan D dengan N sen : $d = n/(1 - z^n)$
- Sebuah bilangan Q dengan D sen : $q = d/(1 - z^d)$
- Sebuah bilangan C dengan Q sen : $c = q/(1 - z^q)$

Dimana {P, N, D, Q, C} adalah set pilihan koin dalam sen sejumlah {1, 5, 10, 25, 50}. Jika kita substitusi untuk menghilangkan denominator maka didapatkan :

- $1 = p * (1 - z)$
- $p = n * (1 - z^5)$
- $n = d * (1 - z^{10})$
- $d = q * (1 - z^{25})$
- $q = c * (1 - z^{50})$

Maka untuk sembarang jumlah koin, kita dapat merepresentasikan sebuah infinite generate function untuk menghitung N dengan X koin yaitu

$$N(z) = \frac{1}{\prod_{i=0}^{|x|} (1 - z^{x_i})}$$

Dimana x_i merupakan elemen ke-i pada set x yang dimiliki oleh soal. Maka untuk soal TPC07 rumus diatas dapat dirubah menjadi

$$N(z) = \frac{1}{(1 - z) * (1 - z^5) * (1 - z^{10}) * (1 - z^{25}) * (1 - z^{50})}$$

Untuk mengefisiensikan perhitungan, kita dapat mengurangi redundansi dengan menghitung repetisi yang terjadi berdasarkan koefisien dari setiap penyebut yang ada pada rumus. Sifat pertumbuhan koefisien akan redundan untuk setiap $LCM(1, 5, 10, 25, 50) = 50$ iterasi yang ada. Berdasarkan teori bilangan kita mendapatkan sifat bilangan bulat yaitu

$$n = pq + r, \{0 < r < \min(p, q) \mid n \in \mathbb{R}\} \leftrightarrow$$

$$n = 50q + r, \{0 < r < \min(q, 50) \mid n \in \mathbb{R}\}$$

Maka kita hanya perlu menghitung setiap kemungkinan r yang ada, dimana komputasi tersebut bisa dilakukan dalam $O(1)$. Maka untuk setiap kemungkinan r yang ada, akan terbentuk sebuah polinomial tetap $f(q)$. Maka untuk n dapat dihitung sebagai $50q + f(q)$. Dikarenakan pada soal terdapat 5 occurrence $(1-z) \rightarrow$ (bisa juga dilihat dari opsi koin yg ada karena rumusnya pasti tetap), maka perhitungan polinomial $f(q)$ akan menjadi polinomial berderajat $4(5-1)$. Maka akan menjadi

$$f(q) = Aq^4 + Cq^3 + Cq^2 + Dq^1 + E$$

Dimana perhitungan koefisien(base point) dapat dilakukan dengan iterasi setiap kemungkinan untuk semua koefisien $f(0), f(1), f(2), f(3), f(4)$ untuk mencari hasil dengan

$$n = \sum_{i=0}^4 r + i * LCM(x)$$

Dimana x adalah jumlah tipe koin yang ada pada soal. Dengan begini kita telah mendapatkan base point(koefisien) untuk sembarang n pada operasi.

Selanjutnya kita akan menghitung $f(q)$. Karena fungsi $f(q)$ tidak terdefinisi untuk sembarang q pada $q \in \mathbb{R}$, maka kita dapat menggunakan **Lagrange interpolation** untuk mengevaluasi nilai $f(q)$ untuk sembarang q . Berdasarkan referensi pada <https://www.geeksforgeeks.org/maths/lagrange-interpolation-formula/>

Didapatkan $f(q)$ menggunakan lagrange interpolation 4th order sebagai berikut :

$$f(q) = \sum_{i=0}^4 y_i \prod_{j=[0,4], j \neq i}^4 \frac{q-j}{i-j}$$

Maka didapatkan $f(q)$ yang berhubungan dengan binomial expansion derajat 4

$$\begin{aligned} f(q) = & \frac{y_0 * (q-1) * (q-2) * (q-3) * (q-4)}{4!} \\ & - \frac{4 * y_1 * q * (q-2) * (q-3) * (q-4)}{4!} \\ & + \frac{6 * y_2 * q * (q-1) * (q-3) * (q-4)}{4!} \\ & - \frac{4 * y_3 * q * (q-1) * (q-2) * (q-4)}{4!} \\ & + \frac{y_4 * q * (q-1) * (q-2) * (q-3)}{4!} \end{aligned}$$

Dengan begini maka perhitungan matematis untuk mencari banyaknya cara menyusun n sen telah terselesaikan. Tetapi untuk menghindari floating point error pada program, kita menghilangkan $\frac{n}{4!}$ Pada setiap operasi dan menghitungnya diakhir dengan membagi hasil akhir menjadi $\text{res}/(4!)$, hal ini valid dikarenakan berdasarkan permintaan soal "banyaknya cara menyusun" dapat dipastikan bahwa $\text{res}/(4!)$ merupakan bilangan bulat, atau $\text{res} \% (4!) = 0$.

4. CODING

Untuk programming menggunakan : C++

Pseudocode :

```
GLOBAL DECLARATION
    dp : array[0..254] of integer

PROCEDURE print(n : 128-bit integer)
    s : array[0..44] of character
    len, i : integer

    if n = 0 then
        print("0\n")
        return

    len <- 0
    while n > 0 do
        s[len] <- character of (n mod 10)
        n <- n div 10
        len <- len + 1

    for i <- len - 1 downto 0 do
        print(s[i])

    print("\n")

PROCEDURE solve()
    n, r : integer
    q, ans : 128-bit integer

    while (read input n) is successful do
        r <- n mod 50
        q <- n div 50

        ans <- 0
        ans <- ans + 1 * dp[r]           * (q-1) * (q-2) * (q-3) * (q-4)
        ans <- ans - 4 * dp[r+50]       * q      * (q-2) * (q-3) * (q-4)
        ans <- ans + 6 * dp[r+100]      * q      * (q-1) * (q-3) * (q-4)
        ans <- ans - 4 * dp[r+150]      * q      * (q-1) * (q-2) * (q-4)
        ans <- ans + 1 * dp[r+200]      * q      * (q-1) * (q-2) * (q-3)

        print(ans div 24)

MAIN PROGRAM
    dp[0] <- 1
    coins : array[0..4] of integer <- [1, 5, 10, 25, 50]
    i, j, c : integer

    for i <- 0 to 4 do
        c <- coins[i]
        for j <- c to 250 do
            dp[j] <- dp[j] + dp[j - c]

    solve()
```

Script :

```
#include <cstdio>
using namespace std;
typedef long long ll;
typedef __int128 int128;
ll dp[255];

void print(int128 n) {
    if (n == 0) {
        printf("0\n");
        return;
    }

    char s[45];
    int len = 0;
    while (n > 0) {
        s[len++] = (char)('0' + (n % 10));
        n /= 10;
    }

    for (int i = len-1; i >= 0; i--) putchar(s[i]);
    putchar('\n');
}

void solve() {
    int n;
    while (scanf("%d", &n) == 1) {
        ll r = n%50;
        int128 q = n/50, ans = 0;

        ans += 1 * dp[r] * (q-1) * (q-2) * (q-3) * (q-4);
        ans -= 4 * dp[r+50] * q * (q-2) * (q-3) * (q-4);
        ans += 6 * dp[r+100] * q * (q-1) * (q-3) * (q-4);
        ans -= 4 * dp[r+150] * q * (q-1) * (q-2) * (q-4);
        ans += 1 * dp[r+200] * q * (q-1) * (q-2) * (q-3);
        print(ans/24);
    }
}

int main() {
    dp[0] = 1;
    int coins[] = {1, 5, 10, 25, 50};
    for (int i = 0; i<5; i++) {
        for (int j = coins[i]; j <= 250; j++) dp[j] += dp[j-coins[i]];
    }
    solve();
    return 0;
}
```

5. PENJELASAN CODING & BUKTI SUBMISSION

Program diatas menggunakan C++, berikut merupakan pembedahan teknis program diatas:

- Perhitungan hasil akhir dilakukan dengan menggunakan int128 yaitu interger berbasis 128 bit yang tersedia di gcc dikarenakan ll tidak dapat menampung hasil
- Void print dibuat untuk memproses int128 agar bisa di print dengan menyimpan digit individual hasil untuk di print dengan looping
- Dp dideklarasikan sebagai global variabel bertipe ll agar dapat memuat prekomputasi dp yang memiliki kemungkinan bernilai $> 10^9$

Berikut merupakan bukti submission :

ID	DATE	PROBLEM	RESULT	TIME	MEM	LANG
35611722	 2026-03-17 06:36:32	Change	accepted edit ideone.it	0.01	5.2M	C++
35611720	 2026-03-17 06:36:19	Change	compilation error edit ideone.it	-	-	C++ 4.3.2
35611704	 2026-03-17 06:33:09	Change	runtime error (SIGXFSZ) edit ideone.it	2.01	5.2M	C++
35611696	 2026-03-17 06:32:12	Change	compilation error edit ideone.it	-	-	C++ 4.3.2
35611653	 2026-03-17 06:26:02	Change	compilation error edit ideone.it	-	-	C++ 4.3.2